Superloop

Spec-Driven Development with Closed-Loop Verification

Brian Fischman | March 2026

AI builds software. AI verifies it works. AI fixes what's broken.

The Closed Loop

No human in the execution path.

BUILD

Agent implements features
from spec, one at a time

VERIFY

Mechanical gates + eval sidecar scores each commit

BOOT & QA

App boots, ACs generated, Playwright validates

DIAGNOSE

Failure catalog + root cause analysis

FIX

Per-failure fix agents with build gates

LEARN — Findings feed back into the next build

The Bottleneck Isn't Code Generation

Today's Workflow

1. Prompt AI to write code
2. Human reviews output
3. Human writes acceptance criteria
4. Human QAs manually against ACs
5. Human prompts for fixes
6. Repeat until it works or you give up

The Constraint

AI produces output faster than humans can verify it. The bottleneck is always human attention — writing ACs, clicking through the app, filing bugs back into prompts.

Scales linearly with human attention. Ceiling = you.

First Successful End-to-End Run

CRE Lease Tracker | 3 features | March 5, 2026

36min
Total Runtime

$6.95
API Cost

29/32
Criteria Passed

3/3
Bugs Auto-Fixed

Phase	Duration	Result	Cost
0: Bootstrap	18s	RUNTIME_READY	—
1: Discovery	2.8 min	9 routes found	$0.46
2a: AC Writer	2.7 min	5 features, 32 criteria	$0.38
2b: Gap Detection	instant	32 criteria	—
3: Playwright	22 min	29 pass, 3 fail, 0 blocked	$4.30
4: RCA	17s	3 root causes (high confidence)	$0.07

What the AI Found and Fixed

All 3 failures identified, diagnosed, fixed, and revalidated autonomously.

RC-001: Export CSV ignored active filters

Exported full dataset instead of filtered results. Fix: wired filter params to export API call.

Applied Manhattan filter (22 results), exported CSV, verified all 22 rows match.

`Files: Comps.tsx, comps.ts`

RC-002: Missing 404 page

Unknown routes silently redirected to /dashboard. Fix: added catch-all route with NotFound component.

Navigated to /this-route-does-not-exist, rendered 404 page with Return to Dashboard link.

`Files: App.tsx`

RC-003: Console 401 error on login page

Auth context fired token-refresh probe on public routes. Fix: skip probe on /login, /register.

Fresh browser on /login, zero console errors after 3-second settle.

`Files: AuthContext.tsx`

Run 1 vs Run 2: Fixes Validated

Run 1 (before fixes)

14 passed
3 failed
8 blocked (infra parse error)
0 of 4 fixes succeeded

Fix agents identified correct files but build gate failed (monorepo bug in Phase 5).
8 criteria blocked by output parse error.

Run 2 (after pipeline fixes)

29 passed
3 failed
0 blocked
3 of 3 fixes succeeded

Every failure diagnosed with high-confidence RCA. Fix agents modified correct files, passed build gates, revalidated via Playwright.

The Industry Ships Pieces. This Is the Full Chain.

Devin / OpenHands / Claude Code	Build agents. Write code.	No runtime validation, no QA, no closed loop.
Playwright Agents (1.56+)	Test agents. Self-heal selectors.	Don't build the app or generate ACs from specs.
GitHub Spec Kit	Spec decomposition. Task plans.	Doesn't validate outcomes. Explicit 'gap in functionality'.
TestSprite / mabl	AI QA platforms. Test and heal.	Don't build features or close the loop back to build.

auto-sdd

Spec → Build → Eval → Boot → QA → RCA → Fix → Revalidate → Learn

What This Means at Scale

Traditional Team

28-feature production app:

6 engineers + PM + QA

$400-600K fully loaded

3-6 months

If scope creeps, double it.

auto-sdd

Same 28-feature app:

1 person + AI pipeline

$50-150 in API costs

Hours, not months

Gets smarter with each campaign.

The constraint shifts from engineering headcount to spec quality.

If you can write clear specs, the system can implement them.

What's Next

Campaign Intelligence

Vector store accumulating build + runtime data across campaigns. Statistical pattern detection warns the next build about likely failures before they happen.

Seed Data Packs

New users start with curated learnings from previous campaigns instead of cold-starting. The system ships pre-trained, not empty.

Larger Campaign

Run a 28-feature campaign end-to-end. Validate scaling assumptions against a production-size application.

Build. Verify. Fix. Learn.

Everything else is demos.

github.com/fischmanb/superloop